

Antony Polukhin <[antoshkka@gmail.com](mailto:antoshkka@gmail.com)>, <[antoshkka@yandex-team.ru](mailto:antoshkka@yandex-team.ru)>

Date: 2016-03-20

# A Proposal to add Attribute `[[visible]]`

## I. Introduction and Motivation

Consider the following shared library source code:

```
// libgrph.so

// Public interface
bool grph_is_tree(graph* g);
bool grph_is_directed(graph* g);

// For internal usage
void in_depth_visitor(graph* g, function<void(graph::node*)>);
void in_breadth_visitor(graph* g, function<void(graph::node*)>);
```

Current C++ Standard lacks an ability to specify, which entities will be exported by shared libraries. Different compilers add extensions to do so, which forces the user to write macro:

```
// libgrph.so

#ifdef _MSC_VER
# define EXPORT __declspec(dllexport)
#else
# define EXPORT __attribute__((visibility("default")))
#endif

// Public interface
EXPORT bool grph_is_tree(graph* g);
EXPORT bool grph_is_directed(graph* g);

// For internal usage
void in_depth_visitor(graph* g, function<void(graph::node*)>);
void in_breadth_visitor(graph* g, function<void(graph::node*)>);
```

Things are getting even worse if the same header file is used during library compilation and as the header that declares user interface:

```
// grph.h

#ifdef EXPORTING
# if _MSC_VER
# define API __declspec(dllexport)
# else
# define API __attribute__((visibility("default")))
# endif
#else
# if _MSC_VER
# define API __declspec(dllimport)
# else
# define API
# endif
#endif

// Public interface
API bool grph_is_tree(graph* g);
API bool grph_is_directed(graph* g);
```

This wording attempts to fix issues mentioned above by providing a single `[[visible]]` attribute:

```
// grph.h

// Public interface
[[visible]] bool grph_is_tree(graph* g);
[[visible]] bool grph_is_directed(graph* g);
```

## II. Impact on the Standard

This proposal does not propose changes to existing headers. It does require change in the core language by adding a new `[[visible]]` attribute.

## III. Design Decisions

### A. Single attribute for import and export.

Having a single attribute will simplify user's code, all the import and export related macro will be gone.

It has been publicly mentioned that with the modules feature, having two attributes (`__declspec(dllexport)` and `__declspec(dllimport)`) is no longer the case for MSVC. Entities need to be marked as being available for DLL linkage using single attribute and then the linker automatically exports entities in modules linked into the library and imports entities otherwise.

Two attributes must not be the case for GCC too, where it is allowed to use `__attribute__((visibility("default")))` on import and export.

### B. Attribute name.

Attribute name in this proposal is neither `[[import]]` nor `[[export]]`. Such names may cause confusion with keywords from modules proposal. Name of the attribute in this proposal is based on GCC's attribute name `visibility`.

### C. Analysis of existing practices.

According to [A. Visibility of entities depending on `\_\_visible\_\_`, linkage and inline](#) existing practice is following:

- export entity from shared libraries if the entity has external linkage, not an inline function and marked with compiler specific visibility attribute
- apply visibility attribute to all the entities of the namespace if namespace has compiler specific visibility attribute
- export all the class member functions, nested classes, static variables and vtable if class has compiler specific visibility attribute
- do not export base (or derived) class of a class with compiler specific visibility attribute if base (or derived) class does not have compiler specific visibility attribute

This proposal attempts to standardize existing practice which is reasonable and well thought through:

- Entity with internal linkage must not become visible. Making it visible change entity's linkage, entity becomes visible by other translation units, resulting in external linkage; such behavior affects optimizer and program performance.
- Inline functions must not become visible. Let's assume that we have a header file with following code:

```
class __visible__ foo {
    void do_something();

public:
    void do1() {
```

```
do_something();
}
};
```

Usually the same header is used by the shared library and by the program that uses the shared library. In that case program that uses shared library will have the `do1()` function inlined. It means that `do1()` from shared library won't be used by program and there's no need to keep that function in shared library.

- Visibility of class members must not depend on access modifiers. Let's assume that we have a header file with following code:

```
class __visible__ foo {
    void do_something();

public:
    void do1() {
        do_something();
    }
};
```

Call to inlined function `do1()` results in calling the private `do_something()` function, so `do_something()` function must be visible in shared library.

- Nested classes of visible class become visible. Here `class` behaves as a namespace, this is an expected behavior.
- Keeping minimal amount of entities exported positively affects shared library load times and compiler optimizations. (If entity with external linkage is not exported and user is building a shared library, then the entity may be assumed as internal for the shared library and more aggressive optimizations may be applied, including removal of unreferenced entity.)

#### D. Silently ignore visibility attribute on platforms that do not support it.

Silent ignorance of visibility attribute does not cause hard detectable errors on the platforms that do not support obtaining external references at program startup from library file:

- attempt to make runtime loadable library will result in error, as platform incapable of handling such libraries
- attempt to make a program that relates on a runtime loadable library will result in linkage error, as external references won't be satisfied

Also consider the case, when we have a source file that is used for producing shared and static libraries. If platform has no support for shared libraries we must not break the build of static libraries, so the visibility attribute must be silently ignored in that case.

#### E. Minimal modifications for the N4567 wording.

There have been a lot of shared library related proposals:

- [N1400: Toward standardization of dynamic libraries](#)
- [N1418: Dynamic Libraries in C++](#)
- [N1428: Draft Proposal for Dynamic Libraries in C++](#)
- [N1496: Draft Proposal for Dynamic Libraries in C++ \(Revision 1\)](#)
- [N1976: Dynamic Shared Objects: Survey and Issues](#)
- [N2015: Plugins in C++](#)
- [N2407: C++ Dynamic Library Support](#)

The proposal you're reading attempts to avoid issues of the proposals from above, minifying the changes to the N4567 wording. The proposal you're reading:

- does not modify Basic Concepts [basic]
- does not add new keywords or terms
- attempts to standardize existing practice by unifying vendor specific extensions
- does not add platform requiremnt for shared libraries suport

## IV. Proposed wording relative to N4567

### 7.6.6 Visible attribute

[**dcl.attr.visible**]

The *attribute-token* `visible` specifies that an entity with external linkage [basic.link] may be obtained:

- by current translation unit at program startup from a library file if the entity has no definition in current translation unit. [*Note*: Attribute `visible` does not satisfy external reference to entity, only provides information that the entity is available to obtain at runtime from library file. - *end note*]
- by other translation units at runtime or at program startup if the entity has definition in current translation unit. [*Note*: In that case `visible` attribute prevents optimizing compilers from removing entity from program and from changing entity type or signature. - *end note*]

On the platforms that do not support obtaining external references at program startup from library files the `visible` attribute may be silently ignored. [*Note*: Program that relies on that attribute shall fail to link because of unsatisfied external references. - *end note*]

`visible` shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. The attribute may be applied to following entities with external linkage:

- to the *declarator-id* in a non-inline function,
- to the *declarator-id* in a non-inline static member function,
- to a variable,
- to a non-anonymous namespace,
- to a static class data member,
- to the declaration or definition of a class (in an elaborated-type-specifier (7.1.6.3) or class-head (Clause 9), respectively)
- to the declaration or definition of an enumeration with type-specifier-seq (in an opaque-enum-declaration or enum-head, respectively (7.2)).

If namespace has the `visible` attribute, then all the entities with external linkage (except inline functions) of the nemaspace shall be implicitly declared with `visible` attribute.

If class is declared or defined with `visible` attribute, then the all the static class data members, non-inline member functions, nested classes and implementation generated type information of the class shall be implicitly declared with `visible` attribute.

The first declaration shall specify the `visible` attribute if any other declaration specifies the `visible` attribute. If an entity is declared with the `visible` attribute in one translation unit and the same entity is declared without the `visible` attribute in another translation unit, the program is ill-formed; no diagnostic required.

## V. Feature-testing macro

For the purposes of SG10 it is sufficient to check for attribute `[[visible]]` using `__has_cpp_attribute`.

## V. Revision History

Revision 0:

- Initial proposal

## VI. References

[DISCUSSION] "[std-proposals] Attributes useful for shared objects (import, export, symbol locations)" discussion. Available online at [https://groups.google.com/a/isocpp.org/forum/?fromgroups#!topic/std-proposals/jzc7d\\_MQUZg](https://groups.google.com/a/isocpp.org/forum/?fromgroups#!topic/std-proposals/jzc7d_MQUZg)

[N4567] Working Draft, Standard for Programming Language C++. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf>

[N1400] Toward standardization of dynamic libraries. Available online at <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2002/n1400.html>

[N1418] Dynamic Libraries in C++. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1418.html>

[N1428] Draft Proposal for Dynamic Libraries in C++. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1428.html>

[N1496] Draft Proposal for Dynamic Libraries in C++ (Revision 1). Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1496.html>

[N1976] Dynamic Shared Objects: Survey and Issues. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1976.html>

[N2015] Plugins in C++. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n2015.pdf>

[N2407] C++ Dynamic Library Support. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2407.html>

VII. Acknowledgements

Sean Middleditch gave very useful comments and suggestions about using a single attribute name for import and export.

VIII. Appendix

A. Visibility of entities depending on \_\_visible\_\_, linkage and inline

Table was generated by program from [C. Program to generate visibility table](#), from [B. Library source file](#)

| Entity Name                                                        | GCC-4.6<br>Import<br>Visibility | GCC-4.8<br>Import<br>Visibility | GCC-5.3<br>Import<br>Visibility | Clang-<br>3.5<br>Import<br>Visibility | Calculated<br>Import<br>Visibility | = | External<br>Linkage | &&<br>Inline<br>function | &&<br>or in<br>__visible_<br>class |
|--------------------------------------------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------------|------------------------------------|---|---------------------|--------------------------|------------------------------------|
| <anonymous>::function                                              | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::function_inline                                       | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::function_static                                       | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::private_member_function                   | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::private_member_function_inline            | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::some_class::private_member_variable                   | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::private_static_member_function            | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::private_static_member_function_inline     | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::some_class::private_static_member_variable            | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::public_member_function                    | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::public_member_function_inline             | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::some_class::public_member_function_inline_not_in_body | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::some_class::public_member_variable                    | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::public_static_member_function             | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::some_class::public_static_member_function_inline      | False                           | False                           | False                           | False                                 | False                              | = | False               | && False                 | && True                            |
| <anonymous>::some_class::public_static_member_variable             | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::variable_extern                                       | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::variable_noextern                                     | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::variable_noextern_const                               | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| <anonymous>::variable_static                                       | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && True                            |
| ::base_nonvis_class::private_member_function                       | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::base_nonvis_class::private_member_function_inline                | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::base_nonvis_class::private_member_variable                       | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && False                           |
| ::base_nonvis_class::private_static_member_function                | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::base_nonvis_class::private_static_member_function_inline         | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::base_nonvis_class::private_static_member_variable                | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::base_nonvis_class::public_member_function                        | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::base_nonvis_class::public_member_function_inline                 | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::base_nonvis_class::public_member_function_inline_not_in_body     | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::base_nonvis_class::public_member_variable                        | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && False                           |
| ::base_nonvis_class::public_static_member_function                 | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::base_nonvis_class::public_static_member_function_inline          | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::base_nonvis_class::public_static_member_variable                 | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::private_member_function                    | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::private_member_function_inline             | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::derived_nonvis_class::private_member_variable                    | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && False                           |
| ::derived_nonvis_class::private_static_member_function             | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::private_static_member_function_inline      | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::derived_nonvis_class::private_static_member_variable             | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::public_member_function                     | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::public_member_function_inline              | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::derived_nonvis_class::public_member_function_inline_not_in_body  | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::derived_nonvis_class::public_member_variable                     | False                           | False                           | False                           | False                                 | False                              | = | False               | && True                  | && False                           |
| ::derived_nonvis_class::public_static_member_function              | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::derived_nonvis_class::public_static_member_function_inline       | False                           | False                           | False                           | False                                 | False                              | = | True                | && False                 | && False                           |
| ::derived_nonvis_class::public_static_member_variable              | False                           | False                           | False                           | False                                 | False                              | = | True                | && True                  | && False                           |
| ::function                                                         | True                            | True                            | True                            | True                                  | True                               | = | True                | && True                  | && True                            |

[illegible]

[illegible]

### B. Library source file

```

main.cpp
#define __visible__ __attribute__((visibility("default")))

#define NONCLASS_ENTITIES(prefix)
/*variables*/
static __visible__ int prefix ## __variable_static = 0;
__visible__ int prefix ## __variable_noextern = 0;
__visible__ const int prefix ## __variable_noextern_const = 0;
extern __visible__ int prefix ## __variable_extern;
int prefix ## __variable_extern = 0;

/*functions*/
__visible__ inline int prefix ## __function_inline() { return 0; };
__visible__ static int prefix ## __function_static() { return 0; };
__visible__ int prefix ## __function() { return 0; };
/**/

#define CLASS_MEMBER_ENTITIES(prefix)
public:
static int prefix ## __public_static_member_function_inline() { return 0; };
static int prefix ## __public_static_member_function();
static int prefix ## __public_static_member_variable;
int prefix ## __public_member_function_inline() { return 0; };
inline int prefix ## __public_member_function_inline_not_in_body();
int prefix ## __public_member_function();
int prefix ## __public_member_variable;

private:
static int prefix ## __private_static_member_function_inline() { return 0; };
static int prefix ## __private_static_member_function();
static int prefix ## __private_static_member_variable;
int prefix ## __private_member_function_inline() { return 0; };
int prefix ## __private_member_function();
int prefix ## __private_member_variable;
/**/

#define CLASS_MEMBER_ENTITIES_DEFINITION(prefix)
int prefix ## __public_static_member_function() { return 0; };
int prefix ## __public_static_member_variable = 0;
int prefix ## __public_member_function() { return 0; };
int prefix ## __public_member_function_inline_not_in_body() { return 0; };
int prefix ## __private_static_member_function() { return 0; };
int prefix ## __private_static_member_variable = 0;
int prefix ## __private_member_function() { return 0; };
/**/

NONCLASS_ENTITIES(global_ns)

struct base_nonvis_class_impl {
    virtual ~base_nonvis_class_impl(){}
    CLASS_MEMBER_ENTITIES(global_ns_base_nonvis_class)
};

CLASS_MEMBER_ENTITIES_DEFINITION(base_nonvis_class_impl::global_ns_base_nonvis_class)

struct __visible__ some_class: base_nonvis_class_impl {
    CLASS_MEMBER_ENTITIES(global_ns_some_class)

    int foo() {

```

```

        return base_nonvis_class_impl::global_ns__base_nonvis_class__public_member_function();
    }
public:
    struct public_nested_class {
        CLASS_MEMBER_ENTITIES(global_ns__some_class__public_nested_class)
    };

    struct public_nested_class_not_inside;
private:
    struct private_nested_class {
        CLASS_MEMBER_ENTITIES(global_ns__some_class__private_nested_class)
    };
};

struct some_class::public_nested_class_not_inside {
    CLASS_MEMBER_ENTITIES(global_ns__some_class__public_nested_class_not_inside)
};

CLASS_MEMBER_ENTITIES_DEFINITION(some_class::global_ns__some_class)
CLASS_MEMBER_ENTITIES_DEFINITION(some_class::public_nested_class::global_ns__some_class__public_nested_class)
CLASS_MEMBER_ENTITIES_DEFINITION(some_class::private_nested_class::global_ns__some_class__private_nested_class)
CLASS_MEMBER_ENTITIES_DEFINITION(some_class::public_nested_class_not_inside::global_ns__some_class__public_nested_class_not_inside)

struct derived_nonvis_class_impl: some_class {
    CLASS_MEMBER_ENTITIES(global_ns__derived_nonvis_class)
};
CLASS_MEMBER_ENTITIES_DEFINITION(derived_nonvis_class_impl::global_ns__derived_nonvis_class)

namespace /*anonymous_ns*/ {
    NONCLASS_ENTITIES(anonymous_ns)
    struct __visible__ anonymous_ns_class {
        CLASS_MEMBER_ENTITIES(anonymous_ns__some_class)
    };
    CLASS_MEMBER_ENTITIES_DEFINITION(anonymous_ns_class::anonymous_ns__some_class)
}

namespace visible_ns __visible__ {
    NONCLASS_ENTITIES(visible_ns)
    struct __visible__ visible_ns_class {
        CLASS_MEMBER_ENTITIES(visible_ns__some_class)
    };
    CLASS_MEMBER_ENTITIES_DEFINITION(visible_ns_class::visible_ns__some_class)

    namespace nested {
        NONCLASS_ENTITIES(visible_ns__nested_ns)
        struct __visible__ visible_ns_nested_ns_class {
            CLASS_MEMBER_ENTITIES(visible_ns__nested_ns__some_class)
        };
        public:
            struct public_nested_class {
                CLASS_MEMBER_ENTITIES(visible_ns__nested_ns__some_class__public_nested_class)
            };

            struct public_nested_class_not_inside;

            struct visible_ns_nested_ns_class::public_nested_class_not_inside {
                CLASS_MEMBER_ENTITIES(visible_ns__nested_ns__some_class__public_nested_class_not_inside)
            };

            CLASS_MEMBER_ENTITIES_DEFINITION(visible_ns_nested_ns_class::visible_ns__nested_ns__some_class)
            CLASS_MEMBER_ENTITIES_DEFINITION(visible_ns_nested_ns_class::public_nested_class::visible_ns__nested_ns__some_class__public_nested_class)
            CLASS_MEMBER_ENTITIES_DEFINITION(visible_ns_nested_ns_class::public_nested_class_not_inside::visible_ns__nested_ns__some_class__public_nested_class_not_inside)
        }
    }
}

```

### C. Program to generate visibility table

```

#!/usr/bin/python

import re
import subprocess

# Compiles the main.cpp, extracts all the exported entities using readelf
def get_exported_entities(compiler, all_entities):
    popen_result = subprocess.Popen(compiler + ' -g -fvisibility=hidden -shared main.cpp -o libmain.so && readelf -sW libmain.so',
        shell=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE)
    popen_result.wait()

    if popen_result.returncode != 0:
        print popen_result.stderr.read()
        exit(popen_result.returncode)

    output = popen_result.stdout
    exported = set()
    while True:
        line = output.readline()
        if not line:
            break

        if not " GLOBAL " in line:
            continue

        exported.update([e for e in all_entities if e in line])

    return exported

def extend_with_colors(items):
    ext = {}
    for key, value in items.iteritems():
        ext[key + "_color"] = 'green' if value else 'red'

    items.update(ext)

# Preprocesses the main.cpp, extracts all the entities with two underlines in the middle
def get_all_entities():
    popen_result = subprocess.Popen('g++ -E main.cpp',
        shell=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE)
    popen_result.wait()

    if popen_result.returncode != 0:
        print popen_result.stderr.read()
        exit(popen_result.returncode)

    output = popen_result.stdout
    all_entities = []
    while True:
        line = output.readline()
        if not line:
            break

        all_entities += re.findall("([_a-z]+__[_a-z]+)", line)

    return list(set(all_entities))

def get_extern_linkage_entities():
    vis_suffix = [
        '_function',
        '__function_inline',
        '_variable_noextern',
        '__variable_extern',

        '__public_member_function',
        '__public_member_function_inline',
        '__public_member_function_inline_not_in_body',
        '__public_static_member_function',
        '__public_static_member_function_inline',

        '_private_member_function',
        '__private_member_function_inline',
        '_private_static_member_function',
        '__private_static_member_function_inline',
    ]

```

```

        '__public_static_member_variable',
        '__private_static_member_variable',
    ]

extern_linkage = set()
for i in vis_suffix:
    if "member" in i:
        extern_linkage.add('global_ns__base_nonvis_class' + i)
        extern_linkage.add('global_ns__some_class' + i)
        extern_linkage.add('global_ns__derived_nonvis_class' + i)
        extern_linkage.add('global_ns__some_class__public_nested_class' + i)
        extern_linkage.add('global_ns__some_class__public_nested_class_not_inside' + i)
        extern_linkage.add('global_ns__some_class__private_nested_class' + i)
        extern_linkage.add('visible_ns__some_class' + i)
        extern_linkage.add('visible_ns__nested_ns__some_class' + i)
        extern_linkage.add('visible_ns__nested_ns__some_class__public_nested_class' + i)
        extern_linkage.add('visible_ns__nested_ns__some_class__public_nested_class_not_inside' + i)
    else:
        extern_linkage.add('global_ns' + i)
        extern_linkage.add('visible_ns' + i)
        extern_linkage.add('visible_ns__nested_ns' + i)

return extern_linkage

def main():
    all_entities = get_all_entities()
    extern_linkage = get_extern_linkage_entities()

    gcc46_exports = get_exported_entities('g++-4.6', all_entities)
    gcc48_exports = get_exported_entities('g++-4.8', all_entities)
    gcc53_exports = get_exported_entities('g++-5', all_entities)
    clang34_exports = get_exported_entities('clang++', all_entities)

    result = []

    pattern = (
        "<tr> "
        "    <td>{name}</td> "
        "    <td bgcolor='{gcc46_color}'>{gcc46}</td> "
        "    <td bgcolor='{gcc48_color}'>{gcc48}</td> "
        "    <td bgcolor='{gcc53_color}'>{gcc53}</td> "
        "    <td bgcolor='{clang34_color}'>{clang34}</td> "
        "    <td style='border:none'> </td> "
        "    <td bgcolor='{computed_color}'>{computed}</td> "
        "    <td>=</td> "
        "    <td bgcolor='{linkage_color}'>{linkage}</td> "
        "    <td>&amp;&amp;</td> "
        "    <td bgcolor='{noinline_color}'>{noinline}</td> "
        "    <td>&amp;&amp;</td> "
        "    <td bgcolor='{visible_color}'>{visible}</td> "
        "</tr>"
    )

    for entity in all_entities:
        not_base_or_derived = not "derived" in entity and not "base" in entity
        is_extern = entity in extern_linkage
        not_inline = not "inline" in entity
        computed_vis = not_base_or_derived and not_inline and is_extern

        s = {
            'name': entity,
            'gcc46': entity in gcc46_exports,
            'gcc48': entity in gcc48_exports,
            'gcc53': entity in gcc53_exports,
            'clang34': entity in clang34_exports,
            'computed': computed_vis,
            'linkage': is_extern,
            'noinline': not_inline,
            'visible': not_base_or_derived,
        }
        extend_with_colors(s)
        result.append( pattern.format(**s) )

    for l in sorted(result):
        print l.replace('_', '::').replace('global_ns', '').replace('anonymous_ns', '&lt;anonymous&gt;')

if __name__ == "__main__":
    main()

```